

PRAKTIKUM SISTEM BASIS DATA 2

STUDI KASUS : Basis Data Sistem GeoHealth Tracker

**Fakultas Teknologi Informasi
Universitas Islam Kalimantan Muhammad Arsyad Al Banjari
Banjarmasin
2026**

PERTEMUAN 1

1. Pendahuluan dan Konteks Sistem

Dalam arsitektur kesehatan masyarakat modern, data bukan sekadar angka; data adalah instrumen penyelamat nyawa. Urgensi sistem "GeoHealth Tracker" terletak pada kemampuannya mengintegrasikan data medis klinis dengan variabel spasial (lokasi). Bagi Dinas Kesehatan, integrasi ini adalah kunci untuk melakukan pengawasan proaktif. Tanpa pemetaan koordinat yang akurat, data imunisasi hanyalah tumpukan baris tabel; namun dengan dimensi geografis, kita dapat memvisualisasikan persebaran cakupan dan mendeteksi "zona merah" atau wilayah rentan yang memerlukan intervensi darurat. Praktikum ini dirancang untuk mentransformasi cara Anda memandang basis data, dari sekadar tempat penyimpanan menjadi mesin analisis yang tangguh. Melalui pengelolaan data kesehatan yang kompleks, Anda akan dilatih untuk membangun sistem yang menjamin integritas informasi sekaligus menyediakan wawasan strategis. Sebelum kita memasuki ranah teknis pengkodean, sangat krusial untuk memahami aturan bisnis dan struktur data yang menjadi fondasi utama sistem ini.

2. Arsitektur Data dan Aturan Bisnis

Dinas Kesehatan Kota ingin memetakan cakupan imunisasi dasar lengkap untuk balita guna mendeteksi wilayah "zona merah" yang rentan wabah. Sistem ini harus mampu menyimpan data medis sekaligus titik koordinat lokasi untuk keperluan visualisasi peta.

Berikut adalah aturan bisnisnya:

1. **Wilayah Administrasi (Kelurahan):** Sistem mencatat Kode Kelurahan (unik), Nama Kelurahan, dan **Poligon Koordinat** wilayah tersebut. Satu kelurahan memiliki banyak balita.
2. **Puskesmas:** Setiap Kelurahan dipantau oleh satu Puskesmas pembina. Data Puskesmas mencakup Kode Puskesmas (unik), Nama Puskesmas, Alamat, dan **Titik Koordinat (Lat/Long)** kantor Puskesmas. Satu Puskesmas bisa membawahi beberapa kelurahan.
3. **Data Balita:** Mencatat NIK Balita (unik), Nama, Tanggal Lahir, Jenis Kelamin, dan Nama Orang Tua. Setiap balita tercatat berdomisili di satu Kelurahan.
4. **Vaksin:** Mencatat Kode Vaksin (unik), Nama Vaksin (misal: BCG, Polio 1, DPT-HB-Hib 1), dan Usia Minimal pemberian.
5. **Riwayat Imunisasi:** Setiap kali balita mendapatkan imunisasi, sistem mencatat No_Transaksi, Tanggal Pemberian, dan Status (Lengkap/Tidak). Satu balita bisa menerima banyak jenis vaksin, dan satu jenis vaksin diberikan

ke banyak balita. Pencatatan ini juga menyimpan informasi di Puskesmas mana vaksin tersebut diberikan.

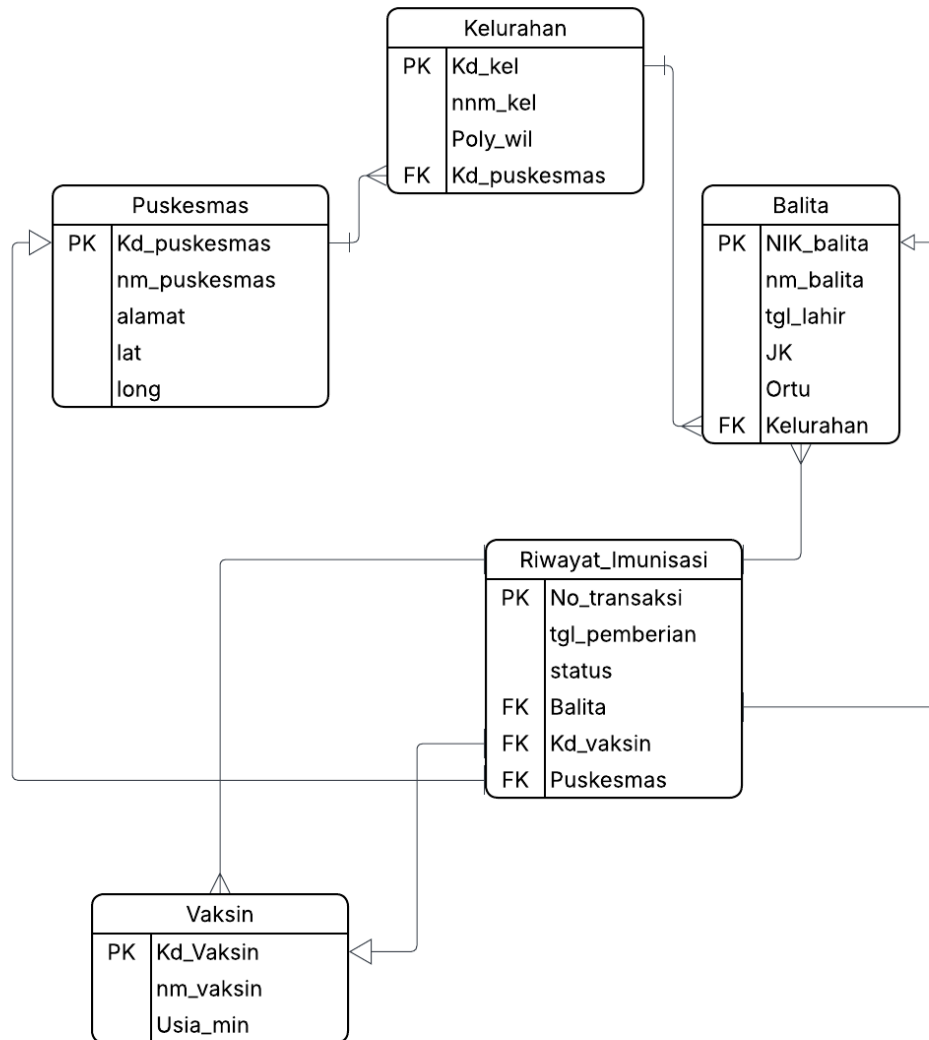
Entitas dan Atribut

Kebutuhan Bisnis	Entitas dan Atribut
Kelurahan: Sistem mencatat Kode Kelurahan (unik), Nama Kelurahan, dan Poligon Koordinat wilayah tersebut.	Tabel Kelurahan (<u>Kd_Kel</u> , nm_kel, poly_wil)
Puskesmas: Data Puskesmas mencakup Kode Puskesmas (unik), Nama Puskesmas, Alamat, dan Titik Koordinat (Lat/Long) kantor Puskesmas.	Tabel Puskesmas (<u>Kd_puskesmas</u> , nm_puskesmas, Alamat, lat, long.
Data Balita: Mencatat NIK Balita (unik), Nama, Tanggal Lahir, Jenis Kelamin, dan Nama Orang Tua.	Tabel Balita (<u>NIK_balita</u> , nm_balita, tgl_lahir, Jk, nm_ortu)
Vaksin: Mencatat Kode Vaksin (unik), Nama Vaksin (misal: BCG, Polio 1, DPT-HB-Hib 1), dan Usia Minimal pemberian	Tabel Vaksin (<u>Kd_vak</u> , nm_vaksin, Usia_min)
Riwayat Imunisasi: sistem mencatat No_Transaksi, Tanggal Pemberian, dan Status (Lengkap/Tidak).	Tabel Riwayat_Imunisasi (<u>No_Transaksi</u> , tgl_pemberian, Status)

Relasi

Kebutuhan Bisnis	Relasi Tabel
Satu kelurahan memiliki banyak balita. Setiap balita tercatat berdomisili di satu Kelurahan.	Kelurahan → Balita 1 : N Tabel Balita (<u>NIK_balita</u> , nm_balita, tgl_lahir, Jk, nm_ortu, Kd_kel)
Setiap Kelurahan dipantau oleh satu Puskesmas pembina. Satu Puskesmas bisa membawahi beberapa kelurahan.	Kelurahan → puskesmas N : 1 Tabel Kelurahan (<u>Kd_Kel</u> , nm_kel, poly_wil, Kd_pus)
Satu balita bisa menerima banyak jenis vaksin, dan satu jenis vaksin diberikan ke banyak balita. Pencatatan ini juga menyimpan informasi di Puskesmas mana vaksin tersebut diberikan	Balita → vaksin M : N Tabel Riwayat_Imunisasi (<u>No_Transaksi</u> , tgl_pemberian, Status, NIK_Balita, Kd_Vaksin, Kd_Puskesmas)

Skema



3. Implementasi Struktur Tabel (DDL) dan Data Sampel (DML)

Data Definition Language (DDL) adalah fondasi di mana integritas data ditegakkan melalui *constraints* dan tipe data yang tepat. Setelah struktur terbentuk, Data Manipulation Language (DML) digunakan untuk mengisi sistem dengan data simulasi yang valid guna menguji ketajaman query kita.

Data Definition Language (DDL)

```
CREATE TABLE puskesmas (  
    kd_puskesmas CHAR(5) PRIMARY KEY,  
    nm_puskesmas VARCHAR(25),  
    alamat VARCHAR(50),  
    lat DECIMAL(10,6),  
    long DECIMAL(10,6)  
);
```

```
CREATE TABLE kelurahan (
  kd_kel CHAR(5) PRIMARY KEY,
  nm_kel VARCHAR(50),
  poly_wil TEXT,
  kd_puskesmas CHAR(5),
  FOREIGN KEY (kd_puskesmas) REFERENCES puskesmas(kd_puskesmas)
);
```

```
CREATE TABLE balita (
  nik_balita CHAR(10) PRIMARY KEY,
  nm_balita VARCHAR(50),
  tgl_lahir DATE,
  jk CHAR(1),
  nm_ortu VARCHAR(50),
  kd_kel CHAR(5),
  FOREIGN KEY (kd_kel) REFERENCES kelurahan(kd_kel)
);
```

```
CREATE TABLE vaksin (
  kd_vaksin CHAR(10) PRIMARY KEY,
  nm_vaksin VARCHAR(50),
  usia_min INT
);
```

```
CREATE TABLE riwayat_imunisasi (
  no_transaksi INT AUTO_INCREMENT PRIMARY KEY,
  tanggal DATE,
  status VARCHAR(10),
  nik_balita CHAR(10),
  kd_vaksin CHAR(10),
  kd_puskesmas CHAR(5),
  FOREIGN KEY (nik_balita) REFERENCES balita(nik_balita),
  FOREIGN KEY (kd_vaksin) REFERENCES vaksin(kd_vaksin),
  FOREIGN KEY (kd_puskesmas) REFERENCES puskesmas(kd_puskesmas)
);
```

Perhatikan penggunaan CHAR pada kolom kode/NIK dan VARCHAR pada kolom nama :

- ✓ Gunakan CHAR untuk data dengan panjang tetap (fixed-length) guna mengoptimalkan kinerja indexing, dan
- ✓ VARCHAR untuk data yang panjangnya bervariasi guna efisiensi penyimpanan.

Data Manipulation Language (DML)

```
-- Insert Data Master
INSERT INTO puskesmas VALUES
('P001','Puskesmas A','Jl. Sehat 1',-6.200000,106.800000),
('P002','Puskesmas B','Jl. Sehat 2',-6.210000,106.810000);

INSERT INTO kelurahan VALUES
('K001','Kelurahan Melati','POLYGON(...)', 'P001'),
('K002','Kelurahan Mawar','POLYGON(...)', 'P002');

INSERT INTO balita VALUES
('B001','Andi','2022-01-01','L','Budi','K001'),
('B002','Siti','2021-05-10','P','Ani','K002'),
('B003','Rina','2022-03-15','P','Dedi','K001');

INSERT INTO vaksin VALUES
('V001','BCG',0),
('V002','Polio',2);

-- Insert Data Transaksi
INSERT INTO riwayat_imunisasi (tanggal, status, nik_balita, kd_vaksin, kd_puskesmas)
VALUES
('2023-01-01','Lengkap','B001','V001','P001'),
('2023-01-10','Lengkap','B002','V002','P002'),
('2023-01-15','Tidak','B003','V001','P001');
```

4. Eksplorasi Query Lanjut: Teknik JOIN

Dalam data kesehatan, pemilihan jenis JOIN menentukan akurasi temuan. INNER JOIN digunakan untuk laporan operasional standar (hanya menampilkan data yang lengkap), sedangkan LEFT JOIN adalah alat utama audit untuk mengidentifikasi celah atau "gap" dalam layanan kesehatan.

Penggunaan INNER JOIN

Query ini menggabungkan data yang memiliki pasangan pasti di kedua tabel, misalnya menampilkan balita yang domisilinya sudah tercatat di kelurahan tertentu.

```
-- Relasi Balita-Kelurahan
SELECT b.nm_balita, k.nm_kel
FROM balita b
INNER JOIN kelurahan k ON b.kd_kel = k.kd_kel;
```

```
-- Multi-Tabel: Riwayat Lengkap per Balita
SELECT b.nm_balita, v.nm_vaksin, p.nm_puskesmas
FROM riwayat_imunisasi r
JOIN balita b ON r.nik_balita = b.nik_balita
JOIN vaksin v ON r.kd_vaksin = v.kd_vaksin
JOIN puskesmas p ON r.kd_puskesmas = p.kd_puskesmas;
```

Identifikasi Gap dengan LEFT JOIN

Untuk menemukan balita yang berisiko tinggi karena belum pernah mendapatkan imunisasi, kita membandingkan tabel balita dengan riwayat_imunisasi.

```
SELECT b.nm_balita
FROM balita b
LEFT JOIN riwayat_imunisasi r ON b.nik_balita = r.nik_balita
WHERE r.nik_balita IS NULL;
```

Mekanisme IS NULL pada kolom foreign key (r.nik_balita) berfungsi sebagai filter eksklusif. Query ini menampilkan semua data dari tabel kiri (balita) dan mencari baris di tabel kanan (riwayat) yang tidak eksis. Celah inilah yang kita sebut sebagai target intervensi medis. Data individual ini kemudian perlu kita agregasikan untuk melihat gambaran besar di tingkat wilayah.

PERTEMUAN 2

Tambahan Data

```
INSERT INTO balita VALUES
```

```
('B004','Dina','2021-08-20','P','Sari','K001'),  
( 'B005','Rudi','2022-06-11','L','Agus','K002'),  
( 'B006','Lina','2021-12-01','P','Wati','K002'),  
( 'B007','Bayu','2022-09-09','L','Joko','K001')  
( 'B008','Nina','2022-02-02','P','Rudi','K001'),  
( 'B009','Fajar','2021-11-11','L','Siti','K002');
```

```
INSERT INTO vaksin VALUES
```

```
('V003','DPT-HB-Hib 1',2),  
( 'V004','Campak',9);
```

```
INSERT INTO riwayat_imunisasi (tanggal, status, nik_balita, kd_vaksin, kd_puskesmas)  
VALUES
```

```
('2023-02-01','Lengkap','B001','V002','P001'),  
( '2023-02-05','Lengkap','B001','V003','P001'),  
( '2023-02-10','Lengkap','B004','V001','P001'),  
( '2023-02-12','Tidak','B005','V002','P002'),  
( '2023-02-15','Lengkap','B006','V001','P002'),  
( '2023-02-20','Lengkap','B006','V003','P002'),  
( '2023-02-25','Lengkap','B006','V004','P002'),  
( '2023-03-01','Lengkap','B008','V001','P001'),  
( '2023-03-05','Lengkap','B008','V002','P001'),  
( '2023-03-10','Tidak','B009','V001','P002');
```

5. Agregasi, Fungsi Statistik, dan Analisis Wilayah

Fungsi agregasi mengubah data mentah menjadi wawasan (*insight*) terukur yang krusial bagi pengambil kebijakan di Dinas Kesehatan untuk menentukan prioritas alokasi sumber daya.

Analisis Cakupan Per Wilayah

Query berikut menghitung populasi balita dibandingkan dengan realisasi imunisasi di setiap kelurahan.

```
SELECT k.nm_kel,  
COUNT(DISTINCT b.nik_balita) AS total_balita,  
COUNT(DISTINCT r.nik_balita) AS sudah_imunisasi  
FROM kelurahan k  
LEFT JOIN balita b ON k.kd_kel = b.kd_kel  
LEFT JOIN riwayat_imunisasi r ON b.nik_balita = r.nik_balita  
GROUP BY k.nm_kel;
```

Penjelasan :

SELECT k.nm_kel,

Bagian ini digunakan untuk mengambil **nama kelurahan** dari tabel kelurahan yang diberi alias k.

- k = alias untuk tabel kelurahan
- nm_kel = nama kelurahan

Artinya, hasil query akan ditampilkan berdasarkan nama kelurahan.

COUNT(DISTINCT b.nik_balita) AS total_balita

Bagian ini digunakan untuk menghitung jumlah seluruh balita yang terdaftar di setiap kelurahan.

Penjelasan:

- b = alias tabel balita
- nik_balita = identitas unik setiap balita
- COUNT() = menghitung jumlah data
- DISTINCT = agar balita yang sama tidak dihitung lebih dari sekali

Alias:

AS total_balita

digunakan untuk memberi nama kolom hasil menjadi **total_balita**.

COUNT(DISTINCT r.nik_balita) AS sudah_imunisasi

Bagian ini digunakan untuk menghitung jumlah balita yang memiliki riwayat imunisasi.

Penjelasan:

- r = alias tabel riwayat_imunisasi
- Jika balita memiliki data pada tabel ini, berarti sudah mendapatkan imunisasi

Alias:

AS sudah_imunisasi

digunakan agar hasil lebih mudah dipahami.

FROM kelurahan k

Menentukan bahwa tabel utama yang digunakan adalah tabel kelurahan.

Artinya, query dimulai dari data seluruh kelurahan.

LEFT JOIN balita b ON k.kd_kel = b.kd_kel

Digunakan untuk menghubungkan tabel kelurahan dengan tabel balita.

Relasi:

- kd_kel pada tabel kelurahan
- kd_kel pada tabel balita

Maknanya:

Setiap balita akan dihubungkan dengan kelurahan tempat tinggalnya.

Karena menggunakan LEFT JOIN, maka:

→ kelurahan yang belum memiliki balita tetap akan muncul dalam hasil query.

LEFT JOIN riwayat_imunisasi r ON b.nik_balita = r.nik_balita

Digunakan untuk menghubungkan tabel balita dengan tabel riwayat_imunisasi.

Relasi:

- nik_balita pada tabel balita
- nik_balita pada tabel riwayat_imunisasi

Maknanya:

Jika balita memiliki data pada tabel riwayat imunisasi, berarti balita tersebut sudah pernah imunisasi.

GROUP BY k.nm_kel;

Digunakan untuk mengelompokkan hasil berdasarkan nama kelurahan.

Artinya:

Setiap kelurahan akan memiliki satu baris hasil yang berisi:

- total balita
- jumlah balita yang sudah imunisasi

Perhitungan Umur Dinamis

Jangan pernah menyimpan umur sebagai kolom statis di tabel. Umur berubah setiap hari. Gunakan fungsi `TIMESTAMPDIFF` untuk menghitung umur secara dinamis saat query dijalankan, memastikan integritas data tetap terjaga sepanjang waktu.

```
SELECT nm_balita,  
TIMESTAMPDIFF(YEAR, tgl_lahir, CURDATE()) AS umur  
FROM balita;
```

Filter Strategis: Identifikasi "Zona Merah"

Kita menggunakan `HAVING` untuk menyaring wilayah yang cakupan imunisasinya belum mencapai 100%.

```
SELECT k.nm_kel  
FROM kelurahan k  
LEFT JOIN balita b ON k.kd_kel = b.kd_kel  
LEFT JOIN riwayat_imunisasi r ON b.nik_balita = r.nik_balita  
GROUP BY k.nm_kel  
HAVING COUNT(r.nik_balita) < COUNT(b.nik_balita);
```

Filter `HAVING` bekerja pada level grup hasil agregasi. Ini lebih efektif daripada `WHERE` dalam konteks ini karena memungkinkan kita membandingkan dua hasil kalkulasi (`COUNT`) secara langsung untuk menentukan wilayah mana yang memiliki kesenjangan layanan. Untuk menyederhanakan akses data bagi pengguna non-teknis, query kompleks ini sebaiknya diabstraksikan melalui `VIEW`.

6. Abstraksi Data dengan Subquery dan VIEW

`VIEW` bertindak sebagai lapisan abstraksi yang meningkatkan keamanan data dengan menyembunyikan kompleksitas tabel asli, sekaligus memudahkan pembuatan laporan yang konsisten.

Validasi Medis dengan Subquery

Subquery memungkinkan kita melakukan pencarian berlapis, baik menggunakan operator `NOT IN` maupun subquery ter-relasi (correlated subquery).

Mencari Balita yang BELUM PERNAH Imunisasi

```
SELECT nm_balita  
FROM balita  
WHERE nik_balita NOT IN (  
SELECT nik_balita FROM riwayat_imunisasi  
);
```

Balita dengan Minimal 2 Imunisasi

```
SELECT b.nm_balita
FROM balita b
WHERE (
    SELECT COUNT(*)
    FROM riwayat_imunisasi r
    WHERE r.nik_balita = b.nik_balita
) >= 2;
```

Berbeda dengan subquery standar, *correlated subquery* merujuk pada kolom dari tabel di luar subquery (b.nik_balita). Mesin database akan mengevaluasi subquery tersebut untuk setiap baris di tabel luar, memungkinkan validasi status imunisasi individu secara sangat spesifik.

Subquery + Agregasi

Kelurahan dengan Imunisasi di Bawah Rata-rata.

```
SELECT nm_kel
FROM kelurahan
WHERE kd_kel IN (
    SELECT k.kd_kel
    FROM kelurahan k
    LEFT JOIN balita b ON k.kd_kel = b.kd_kel
    LEFT JOIN riwayat_imunisasi r ON b.nik_balita = r.nik_balita
    GROUP BY k.kd_kel
    HAVING COUNT(r.nik_balita) < (
        SELECT AVG(jumlah)
        FROM (
            SELECT COUNT(r2.nik_balita) AS jumlah
            FROM kelurahan k2
            LEFT JOIN balita b2 ON k2.kd_kel = b2.kd_kel
            LEFT JOIN riwayat_imunisasi r2 ON b2.nik_balita = r2.nik_balita
            GROUP BY k2.kd_kel
        ) AS sub
    )
);
```

Implementasi VIEW untuk Pelaporan

VIEW merangkum logika join dan agregasi yang rumit menjadi satu tabel virtual yang siap pakai.

- **v_imunisasi:** Menyediakan laporan komprehensif yang menggabungkan nama balita, kelurahan, jenis vaksin, dan status pemberian dalam satu tampilan.
- **v_zona_merah (View Analitik):** Secara otomatis menghitung total balita dan total imunisasi per kelurahan untuk memantau efektivitas program kesehatan di tingkat lokal.

```
-- VIEW: Detail Laporan Imunisasi
```

```
CREATE VIEW v_imunisasi AS
SELECT b.nm_balita, k.nm_kel, v.nm_vaksin, r.status
FROM riwayat_imunisasi r
JOIN balita b ON r.nik_balita = b.nik_balita
JOIN kelurahan k ON b.kd_kel = k.kd_kel
JOIN vaksin v ON r.kd_vaksin = v.kd_vaksin;
```

```
-- VIEW: Analitik Zona Merah
```

```
CREATE VIEW v_zona_merah AS
SELECT k.nm_kel,
COUNT(b.nik_balita) AS total,
COUNT(r.nik_balita) AS imunisasi
FROM kelurahan k
LEFT JOIN balita b ON k.kd_kel = b.kd_kel
LEFT JOIN riwayat_imunisasi r ON b.nik_balita = r.nik_balita
GROUP BY k.nm_kel;
```

PERTEMUAN 3

Tambahan Data

```
INSERT INTO balita VALUES
('B010','Rahma','2022-04-12','P','Sulastri','K001'),
('B011','Yoga','2021-07-17','L','Hendra','K002'),
('B012','Alya','2022-10-05','P','Nina','K001'),
('B013','Rizki','2021-09-30','L','Hasan','K002'),
('B014','Putri','2022-01-25','P','Dewi','K001');

INSERT INTO riwayat_imunisasi
(tanggal, status, nik_balita, kd_vaksin, kd_puskesmas)
VALUES
('2023-03-15','Lengkap','B010','V001','P001'),
('2023-03-18','Lengkap','B010','V002','P001'),
('2023-03-20','Tidak','B011','V003','P002'),
('2023-03-22','Lengkap','B012','V001','P001'),
('2023-03-25','Lengkap','B013','V004','P002'),
('2023-03-28','Lengkap','B014','V002','P001');
```

7. Indexing dan Stored Procedure

Dalam sistem basis data berskala besar seperti GeoHealth Tracker, pencarian data yang cepat sangat penting untuk mendukung pengambilan keputusan secara real-time. Ketika jumlah data balita dan riwayat imunisasi terus bertambah, proses pencarian tanpa optimasi akan menjadi lambat.

Salah satu teknik optimasi yang paling penting adalah penggunaan INDEX.

INDEX bekerja seperti daftar isi pada sebuah buku. Tanpa index, database harus membaca seluruh tabel (Full Table Scan) untuk menemukan data yang dicari. Dengan index, database dapat langsung menuju lokasi data yang dibutuhkan.

Hal ini sangat penting pada sistem kesehatan masyarakat karena laporan imunisasi harus dapat diakses dengan cepat untuk mendeteksi wilayah rawan wabah.

Membuat INDEX

Index pada NIK Balita

Karena pencarian berdasarkan NIK sangat sering dilakukan, maka kolom ini perlu dioptimalkan.

```
CREATE INDEX idx_nik_balita  
ON balita(nik_balita);
```

Index pada Kode Kelurahan

Digunakan untuk mempercepat proses JOIN antara tabel balita dan kelurahan.

```
CREATE INDEX idx_kd_kel  
ON balita(kd_kel);
```

Index pada Tanggal Imunisasi

Membantu mempercepat pencarian data berdasarkan periode waktu tertentu.

```
CREATE INDEX idx_tanggal_imunisasi  
ON riwayat_imunisasi(tanggal);
```

Contoh Query yang Memanfaatkan INDEX

Pencarian Balita Berdasarkan NIK

```
SELECT *  
FROM balita  
WHERE nik_balita = 'B010';
```

Secara konseptual, query tetap dapat dijalankan tanpa INDEX. Namun, perbedaannya terletak pada mekanisme eksekusi query di dalam database engine, khususnya pada aspek efisiensi dan performa.

Pencarian Riwayat Imunisasi Berdasarkan Tanggal

Query ini sangat umum digunakan dalam pembuatan laporan bulanan imunisasi

```
SELECT *  
FROM riwayat_imunisasi  
WHERE tanggal BETWEEN '2023-03-01' AND '2023-03-31';
```

Jika query dijalankan tanpa index:

Maka database akan melakukan Full Table Scan, yaitu:

- membaca seluruh baris pada tabel riwayat_imunisasi
- memeriksa satu per satu apakah memenuhi kondisi WHERE

Ilustrasi Proses, Jika terdapat:

- 1.000 data → membaca 1.000 baris
- 100.000 data → membaca 100.000 baris
- 1 juta data → membaca 1 juta baris

Artinya, waktu eksekusi akan meningkat secara linear terhadap jumlah data.

8. Otomatisasi Proses dengan Stored Procedure

Jika query kompleks digunakan berulang kali, maka penulisan ulang query menjadi tidak efisien dan berisiko menimbulkan kesalahan.

Stored Procedure adalah sekumpulan perintah SQL yang disimpan di dalam database dan dapat dipanggil kembali kapan saja. Stored Procedure membantu:

- mengurangi penulisan query berulang
- meningkatkan konsistensi proses
- meningkatkan keamanan data
- mempercepat eksekusi query

Pada sistem GeoHealth Tracker, Stored Procedure sangat berguna untuk pelaporan rutin dan monitoring cakupan imunisasi.

Stored Procedure 1

Menampilkan Data Imunisasi Berdasarkan Kelurahan

```
DELIMITER //

CREATE PROCEDURE sp_imunisasi_kelurahan (
  IN p_nama_kelurahan VARCHAR(50)
)
BEGIN
  SELECT
    b.nm_balita,
    k.nm_kel,
    v.nm_vaksin,
    r.tanggal,
    r.status
  FROM riwayat_imunisasi r
  JOIN balita b ON r.nik_balita = b.nik_balita
  JOIN kelurahan k ON b.kd_kel = k.kd_kel
  JOIN vaksin v ON r.kd_vaksin = v.kd_vaksin
  WHERE k.nm_kel = p_nama_kelurahan;
END //

DELIMITER ;
```


Cara Menjalankan Procedure

Procedure ini akan menampilkan seluruh riwayat imunisasi balita pada kelurahan tertentu.

```
CALL sp_imunisasi_kelurahan('Kelurahan Melati');
```

Stored Procedure 2

Menghitung Total Balita yang Sudah Imunisasi

```
DELIMITER //

CREATE PROCEDURE sp_total_imunisasi ()
BEGIN
    SELECT
        COUNT(DISTINCT nik_balita) AS total_sudah_imunisasi
    FROM riwayat_imunisasi;
END //

DELIMITER ;
```

Menjalankan Procedure

Procedure ini digunakan untuk pelaporan cepat jumlah balita yang telah mendapatkan imunisasi

```
CALL sp_total_imunisasi();
```

INDEX digunakan untuk meningkatkan performa pencarian data, sedangkan Stored Procedure digunakan untuk mengotomatisasi proses query yang kompleks dan berulang.

PERTEMUAN 4

Pada sistem basis data kesehatan seperti GeoHealth Tracker, integritas data merupakan aspek yang sangat penting. Kesalahan input, perubahan data yang tidak terkontrol, maupun kegagalan sistem dapat menyebabkan hilangnya informasi penting terkait imunisasi balita.

Untuk menjaga konsistensi, keamanan, dan keberlangsungan data, diperlukan mekanisme lanjutan dalam pengelolaan basis data, yaitu:

1. Trigger → untuk otomatisasi proses tertentu saat terjadi perubahan data
2. Backup & Restore → untuk perlindungan data dari kehilangan atau kerusakan

Trigger adalah sekumpulan perintah SQL yang akan dijalankan secara otomatis ketika terjadi event tertentu pada tabel, seperti:

- INSERT
- UPDATE
- DELETE

Trigger sangat berguna untuk menjaga konsistensi data tanpa harus dilakukan secara manual oleh pengguna.

Dalam sistem imunisasi, trigger dapat digunakan untuk:

- mencatat log perubahan data
- validasi otomatis
- update status otomatis
- audit aktivitas database

Studi Kasus Trigger

Ketika data imunisasi baru dimasukkan ke tabel riwayat_imunisasi, sistem secara otomatis akan mencatat aktivitas tersebut ke tabel log.

```
CREATE TABLE log_imunisasi (  
  id_log INT AUTO_INCREMENT PRIMARY KEY,  
  no_transaksi INT,  
  aktivitas VARCHAR(100),  
  waktu TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Membuat Trigger AFTER INSERT

```
DELIMITER //
```

```
CREATE TRIGGER trg_after_insert_imunisasi  
AFTER INSERT ON riwayat_imunisasi  
FOR EACH ROW
```

```

BEGIN
  INSERT INTO log_imunisasi
  (no_transaksi, aktivitas)
  VALUES
  (
    NEW.no_transaksi,
    CONCAT('Data imunisasi baru ditambahkan untuk Balita: ', NEW.nik_balita)
  );
END //

DELIMITER ;

```

Uji Coba Trigger

Setelah query dijalankan, data log akan otomatis masuk ke tabel log_imunisasi.

```

INSERT INTO riwayat_imunisasi
(tanggal, status, nik_balita, kd_vaksin, kd_puskesmas)
VALUES
('2023-04-01','Lengkap','B012','V002','P001');

```

Melihat Hasil Trigger

```

SELECT * FROM log_imunisasi;

```

9. Backup dan Restore

Backup adalah proses membuat salinan data database untuk mengantisipasi:

- kerusakan sistem
- kehilangan data
- serangan malware
- human error
- kegagalan server

Backup adalah prosedur wajib dalam sistem informasi kesehatan.

Backup Database MySQL menggunakan Command Line.

```

mysqldump -u root -p geohealth_tracker > geohealth_backup.sql

```

Penjelasan:

- mysqldump → tool backup MySQL
- geohealth_tracker → nama database
- > → menyimpan hasil ke file SQL

Restore Database MySQL

Mengembalikan Database dari File Backup

Perintah ini akan mengembalikan seluruh isi database dari file backup.

```
mysql -u root -p geohealth_tracker < geohealth_backup.sql
```

Backup Tabel Tertentu

Digunakan jika hanya ingin membackup tabel tertentu saja.

```
mysqldump -u root -p geohealth_tracker balita > backup_balita.sql
```

Praktik Validasi Backup

Setelah restore dilakukan, lakukan pengecekan yang tujuannya untuk memastikan seluruh data berhasil dipulihkan.

```
SELECT COUNT(*) FROM balita;  
SELECT COUNT(*) FROM riwayat_imunisasi;
```